

Laboratório de Concorrência · Aula 10

Gustavo Cardoso · RA 240111752 · Turma ESOF6S-A

Gerado em 09/10/2026, 20:42:17 · tempo registrado no cronômetro: 20:05

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

Cooperação eu quero trabalhar em conjunto com o dado, não quero "passar por cima dele", um exemplo no dia a dia que penso seria algo como um sinal de trânsito, eu preciso que o sinal fique verde (tarefa A) para que o carro possa avançar (tarefa B).

Competição, como no nome já diz, estou disputando o mesmo espaço/recurso, é como se duas pessoas estivessem brigando entre si para pegar um produto da black friday, só existe um e duas pessoas estão lutando por ele.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

algo em torno de 2.000.000

1ª execução (n = 1.000.000)

1034083

2ª execução (n = 1.000.000)

1039436

3ª execução (n = 1.000.000)

1034451

Saída do programa

não respondido

```
1º Execução - 1.000.000:
esperado: 2000000
obtido: 1034083

2º Execução - 1.000.000:
esperado: 2000000
obtido: 1039436

3º Execução - 1.000.000:
esperado: 2000000
obtido: 1034451

1º Execução - 1.000:
esperado: 2000
obtido: 2000

2º Execução - 1.000:
esperado: 2000
obtido: 2000

3º Execução - 1.000:
esperado: 2000
obtido: 2000
```

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não batem com o esperado, todas as execuções foram diferentes umas das outras, a razão para isso acontecer porque as duas threads acessam e modificam a mesma variável ao mesmo tempo, causando uma condição de corrida.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

Ler: a Thread 1 e a Thread 2 leem o mesmo valor do contador, por exemplo, 10.

Somar: cada thread soma 1 ao valor lido, obtendo 11.

Gravar: as duas threads gravam 11 no contador. Embora tenham ocorrido duas tentativas de incremento, o contador passa de 10 para 11, em vez de passar para 12.

1.3 Com $n = 1000$, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com $n = 1000$, o resultado pode ter sido igual ao esperado, 2.000, porque tem menos oportunidades para acontecer conflitos entre as threads. Esse tipo de erro é perigoso porque pode não aparecer durante os testes, mas ocorrer em produção, quando há mais operações simultâneas.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AAABBB → 8

ABABAB → 6

BBBAAA → 7

BBAABA → 4

ABABBA → 4

Saída do programa

```
print("Ordens possíveis:", sum(len(v) for v in resultados.values()))
for valor in sorted(resultados):
    ordens = resultados[valor]
    print(f"TOTAL = {valor}: {len(ordens):2d} ordens, por exemplo {ordens[0]}")
131 ms
AAABBB → 8
BBBAAA → 7
ABABAB → 6
Ordens possíveis: 20
TOTAL = 4: 9 ordens, por exemplo AABBBB
TOTAL = 6: 9 ordens, por exemplo AABABB
TOTAL = 7: 1 ordens, por exemplo BBBAAA
TOTAL = 8: 1 ordens, por exemplo AAABBB
```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

são possíveis 20 ordens diferente mas apenas uma ordem dá o resultado de 8 (AAABBB)

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Na ordem AABBBB, acontece o seguinte:

- A lê o valor 3.
- A calcula $3 + 1 = 4$.
- B lê o valor 3.
- B calcula $3 \times 2 = 6$.
- B grava 6 no TOTAL.
- A grava 4 no TOTAL, sobrescrevendo o valor 6.

O resultado final é 4, porque a última gravação foi feita pela tarefa A. A operação de B foi perdida, pois seu resultado foi sobrescrito quando A gravou o valor que havia calculado anteriormente.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Monitor me parece ser a mais correta e rápida, com a Operação atômica me parecendo que é mais suscetível a erros

Saída do programa

```
esperado:          20000000
sem sincronizacao: 1025844 em 2 ms
synchronized:      20000000 em 86 ms
AtomicInteger:      20000000 em 66 ms
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

synchronized e AtomicInteger, costuma ser mais lento por deve fazer mais passos de segurança para a operação não se perder no meio

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não. Um AtomicInteger em cada saldo garante que cada atualização individual seja atômica, mas não garante que a retirada de uma conta e o depósito na outra aconteçam como uma única operação. Outra thread poderia acessar os saldos entre essas duas etapas, causando inconsistências.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

na teoria acho que nao vão aparecer, já que A vai pegar os dois processos mais nao vai conseguir acaba-los, provavelmente B vai dar erro também

Saídas (antes e depois da correção)

```
primeira saída:  
IMPASSE: 2 threads presas
```

```
segunda saída (corrigida):  
A terminou  
B terminou  
sem impasse
```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

As duas threads ficaram presas porque cada uma segurou um recurso e esperou pelo recurso que estava com a outra. A queria pegar o Y e B queria pegar X, como ambos estavam segurando este no momento, o processo nao executa.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Pegar os recursos sempre na mesma ordem evita que cada thread fique segurando um recurso enquanto espera pelo outro em uma ordem inversa. Assim, se ambas pegarem primeiro X e depois Y, uma delas terá que esperar por X antes de adquirir qualquer outro recurso. Quando a primeira terminar e liberar os recursos, a segunda poderá continuar. Um exemplo seria em um sistema bancário que faz transferências entre contas ao mesmo tempo. Se uma transferência bloquear primeiro a conta A e depois a B, enquanto outra bloquear primeiro B e depois A, poderá ocorrer um impasse.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

acho que levaria 300ms pra ser feita com 3 trabalhadores, com 1 deve ser 600ms e 6 deve ser 100ms ou 50ms (baseado no slide se entendi certo).

Tempo com 1 trabalhador

600ms

Tempo com 3 trabalhadores

200ms

Tempo com 6 trabalhadores

100ms

Saída do programa

1° Print: 1 Trabalhador
2° Print: 3 Trabalhador
3° Print: 6 Trabalhador

```
trabalhador 1 fez a tarefa 1  
trabalhador 1 fez a tarefa 2  
trabalhador 1 fez a tarefa 3  
trabalhador 1 fez a tarefa 4  
trabalhador 1 fez a tarefa 5  
trabalhador 1 fez a tarefa 6  
1 trabalhadores, 6 tarefas de 100 ms: levou 600ms
```

```
trabalhador 1 fez a tarefa 2  
trabalhador 2 fez a tarefa 3  
trabalhador 3 fez a tarefa 1  
trabalhador 3 fez a tarefa 6  
trabalhador 1 fez a tarefa 4  
trabalhador 2 fez a tarefa 5  
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms
```

```
trabalhador 2 fez a tarefa 3  
trabalhador 5 fez a tarefa 6  
trabalhador 1 fez a tarefa 2  
trabalhador 6 fez a tarefa 1  
trabalhador 4 fez a tarefa 5  
trabalhador 3 fez a tarefa 4  
6 trabalhadores, 6 tarefas de 100 ms: levou 100ms
```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

600ms = um trabalhador pegou todas as tarefas e fez uma a uma, sozinho fica 100ms para cada tarefa;

200ms = três trabalhadores pegam aproximadamente duas tarefas para fazerem entre si;

100ms = com todos os trabalhadores pegando uma tarefa, fica mais rápido concluir todas;

O programa não precisa de synchronized nem de travas porque as tarefas são distribuídas por um canal (channel), que permite a comunicação segura entre as goroutines sem precisar compartilhar diretamente uma variável para controlar o trabalho.

Saídas (antes e depois da troca)

não respondido

```
fatal error: all goroutines are asleep - deadlock!

goroutine 1 [chan send]:
main.main()
    /box/main.go:9 +0x38
exit status 2
```

```
1
```

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

O primeiro programa trava porque o canal não possui espaço para armazenar mensagens. Ao executar `ch <- 1`, o programa fica esperando que outra goroutine receba o valor enviado. Como não existe outra goroutine para recebê-lo, ocorre um bloqueio e o programa apresenta um erro de deadlock.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

no mundo ideal acho que somaria a cada iteração, mas provavelmente vai dar erro

Saídas (o erro e o resultado com move)

não respondido

```
error[E0373]: closure may outlive the current function, but it
borrows `contador`, which is owned by the current function
--> HelloWorld.rs:7:27
   |
7 |     let t = thread::spawn(|| {
   |                               ^^ may outlive borrowed value
`contador`
8 |         contador += 1;           // a thread
  altera a variável de main
   |         ----- `contador` is borrowed here
   |
```

```
warning: 2 warnings emitted
```

```
0
```

Saída do programa

```
contador = 400000
```

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Com move, a thread passa a ser dona da variável contador, recebendo o valor inicial 0. Assim, quando a thread executa contador += 1, ela altera sua própria variável, e não a variável original de main. Por isso, ao final, o programa imprime 0.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java, o erro foi achado durante a execução do programa, quando o contador apresentou resultados incorretos. Em Rust, o compilador detecta o problema na compilação e impede que o programa compile enquanto o compartilhamento da variável não estiver correto. A vantagem do Rust é que ele ajuda a evitar erros de concorrência antes que o programa seja executado, aumentando a segurança e a confiabilidade.

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A, D, C, B, E

Saída do programa

não respondido

```
A
D
C
B
E: timer de 10 ms rodou depois de 300 ms
```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

A, D, C, B, E.

A e D foram impressas primeiro porque são instruções síncronas, executadas imediatamente. C veio depois porque as Promises são processadas na fila de microtarefas, antes dos temporizadores. B, mesmo com tempo 0, precisou esperar o código síncrono e as microtarefas terminarem para ser executada. Por isso, saiu depois de D e C.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O timer rodou depois de 10 ms porque a função ocupado(300) bloqueou a única thread por aproximadamente 300 ms. Enquanto esse cálculo estava acontecendo, o laço de eventos não conseguia executar outras tarefas, incluindo o timer.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O programador, com sincronização

Rust com Arc e Mutex

O programador, com sincronização

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Pode ocorrer uma condição de corrida, em que as duas pessoas comprem o mesmo ingresso porque o sistema verifica a disponibilidade antes de registrar as compras. Para evitar isso, usaria uma transação no banco de dados com bloqueio ou uma operação atômica que garanta que apenas uma compra seja confirmada.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

Honestamente a grande diferença de como cada linguagem lida com processos, devo admitir que mesmo lendo o material e até usando a ia ficou estranho de entender.